

RAGE

Retrieval-Augmented Governance Engine

Capa de gobernanza explicable contra prompt injection y jailbreaks en sistemas basados en LLM

Documento de arquitectura del sistema — v2

Rehecho con feedback de asesores, mapeo a OWASP Top 10 LLM 2025, estado del arte y diferenciadores métricos

Proyecto de hackathon · AI Safety

1. Resumen ejecutivo

RAGE es una capa de seguridad que se coloca **delante de un LLM** (chatbots, asistentes y especialmente **agentes conectados a APIs y bases de datos**) para detectar y contener ataques de **prompt injection** y **jailbreak** — el riesgo nº1 del OWASP Top 10 para LLM 2025.

A diferencia de un guardrail que solo cuenta cuántos prompts bloqueó, RAGE aporta: (a) una **arquitectura en capas con motor de decisión explicable**, (b) una **base de conocimiento de amenazas actualizable sin reentrenar**, (c) una **métrica científica de degradación multi-turno (AUC)**, y (d) un **harness de medición reproducible** que cuantifica el riesgo antes/después.

Honestidad técnica: la idea de detección por *retrieval* ya existe en la literatura (RAD, Vigil, Rebuff). El aporte de RAGE es de **integración, gobernanza, explicabilidad y medición**, no un algoritmo nuevo.

2. El problema

Los LLM **no distinguen de forma fiable instrucciones de datos**: cualquier texto en su contexto puede actuar como orden. Esto hace que el prompt injection sea un fallo **arquitectónico**, no un bug que se parchea.

- Sistemas **sin defensa**: tasa de éxito de ataque (ASR) > **90%**.
- **Ninguna defensa es perfecta**: el estudio “*The Attacker Moves Second*” (OpenAI/Anthropic/DeepMind, 2025) rompió 12 defensas publicadas (>90% ASR con ataques adaptativos).
- El consenso del campo es **defensa en profundidad** (varias capas), monitoreo y testing adversarial continuo.

3. Visión de RAGE

Convertir la detección por retrieval en una **capa de gobernanza**: explicable (traza al ataque conocido), **medible** (ASR, severidad, AUC, falsos positivos, costo), **actualizable en caliente** (nuevas amenazas al KB sin reentrenar) e **independiente del modelo y del proveedor** (desplegable on-prem, delante de cualquier LLM).

4. Arquitectura del sistema

RAGE se implementa como una **cascada con salida temprana (early-exit)**, ordenada de lo más barato/preciso a lo más caro. La mayoría del tráfico se resuelve en etapas baratas; el clasificador LLM solo se invoca en casos ambiguos. Esto mantiene el sobre costo bajo (~+1–15%).



Notas de diseño clave:

- El **contenido recuperado** y el **micro-resumen** son **texto influido por el atacante** → se tratan como **no confiables** (mitiga LLM08 y la inyección recursiva).
- El **umbral** del motor de decisión es configurable → controla el balance seguridad ↔ utilidad (como en RAD).
- La **verificación de salida** atrapa ataques que pasaron las capas previas (~60% de los que penetran, según la literatura).

5. Diferenciadores clave

Los tres se conectan: **B** es el mecanismo, **A** es la métrica que lo prueba a lo largo del tiempo, y **C** es el escenario grave que lo hace impactante.

A. Métrica: Área Bajo la Curva (AUC) de degradación multi-turno

Se grafica el **score de vulnerabilidad (eje Y)** contra los **turnos de la conversación (eje X)** y se integra con la regla del trapecio para obtener un **número único**. AUC bajo = la defensa contuvo el ataque a lo largo del tiempo; AUC alto = el guardrail tradicional colapsó en turnos avanzados.

Blindajes metodológicos (imprescindibles):

- **Sin circularidad:** el eje Y debe venir de **ground truth** (¿el sistema protegido realmente filtró el canario o ejecutó la acción prohibida?), NO del score que RAGE se asigna a sí mismo.
- **Normalizar** el AUC (dividir entre el área máxima = score_máx × nº turnos) para comparar conversaciones de distinta longitud.
- Graficar **dos curvas** en el mismo ataque (sin defensa vs RAGE) y reportar el **turno de compromiso** (cuándo cruza el umbral por primera vez).

B. Filtro semántico dinámico (contextual, con estado)

En vez de buscar solo palabras prohibidas, evalúa el **cambio de intención respecto al turno anterior** mediante un micro-resumen de la conversación: ¿el usuario cambia radicalmente de tema o intenta que el agente ignore sus reglas corporativas? Esto convierte la defensa en una **barrera contextual, no solo semántica** — justo el hueco donde fallan los guardrails de un solo turno.

- **Costo:** primero detectar drift por embeddings (barato) y solo disparar el resumen/clasificador LLM cuando hay sospecha (encaja en la cascada).
- El micro-resumen es **no confiable** (texto del atacante) → debe aislarse.

C. Agentes empresariales conectados (caso grave)

La mayoría de benchmarks prueban chats puros. RAGE defiende un **agente con herramientas conectadas a API/DB**. Demo: una inyección que intenta ejecutar **DROP TABLE** o exfiltrar datos sobre una **base de ventas falsa (SQLite en memoria)**. Mapea a OWASP LLM06 (**Excessive Agency**) y LLM05.

- **Defensa en capas a nivel de acción:** aunque la inyección llegue al agente, el gateway gatea la tool-call (allowlist, solo SELECT parametrizado; bloquea DROP/DELETE/UPDATE).
- Aquí el **canario es una acción** (“¿se ejecutó DROP TABLE?”), lo que alimenta directo el eje Y de ground truth de la métrica (A).

6. Mapeo a OWASP Top 10 para LLM 2025

Riesgo OWASP 2025	¿RAGE lo aborda?	Cómo
LLM01 Prompt Injection	Sí, hoy (núcleo)	Cascada de detección + decisión bloquea inyección directa, indirecta, payload splitting, ofuscación
LLM07 System Prompt Leakage	Sí, hoy	Detecta intentos de filtrar instrucciones/secretos (lo mide el harness con canarios)
LLM02 Sensitive Info Disclosure	Con cambios	Filtro de salida que redacta/bloquea fugas (PII, secretos)
LLM05 Improper Output Handling	Con cambios	Capa de validación/sanitización de salida
LLM06 Excessive Agency	Con cambios	Motor de decisión sobre tool-calls (block / human-in-the-loop)
LLM09 Misinformation	Con cambios	Verificación de groundedness (RAG Triad)
LLM08 Vector & Embedding Weaknesses	Riesgo que RAGE introduce	RAGE usa vector store → proteger integridad y acceso del KB
LLM03 / LLM04 / LLM10	Fuera de alcance	Cadena de suministro, envenenamiento de entrenamiento, consumo (parcial: rate-limit)

7. Estado del arte y aporte honesto de RAGE

Familia de defensa	Ejemplos	Adaptable sin reentrenar	Explicable
Prompting / system prompt	Instruction hierarchy, spotlighting	n/a	No
Fine-tuning	StruQ (~45% ASR), SecAlign (~8%)	No (reentrenar)	No
Clasificadores ML	Prompt Guard, Llama Guard, Vigil	No (reentrenar)	Limitada
Servicio gestionado	Lakera Guard (<50ms)	Parcial	Dashboard
Frameworks	NeMo Guardrails, Guardrails AI, LLM Guard, Rebuff	Parcial	Parcial
Arquitectónicos	CaMeL, dual-LLM, structured queries	n/a	Parcial
RAG-based (familia de RAGE)	RAD, RePD, Vigil	Sí (al KB)	Sí

Prior art: el concepto de RAGE ya existe (p. ej. **RAD**, arXiv 2508.16406: RAG + umbral ajustable + KB sin reentrenar). **RAGE no inventa el mecanismo**; su valor es volverlo una capa de gobernanza explicable, medible (incl. AUC multi-turno), mapeada a OWASP y aplicada a agentes conectados.

8. Pregunta de investigación e hipótesis

Pregunta principal: ¿En qué medida una capa de gobernanza basada en RAG (intención + recuperación de amenazas + decisión) reduce la tasa de éxito de prompt injection (OWASP LLM01) en agentes basados en LLM, y a qué costo de latencia/cómputo y de falsos positivos, comparada con las defensas actuales?

- **H1 (efectividad):** RAGE reduce significativamente el ASR vs el baseline de solo system prompt.
- **H2 (costo):** lo logra con sobrecosto bajo (~+1–15%) y latencia tolerable.
- **H3 (utilidad):** mantiene baja la tasa de falsos positivos (no rompe tráfico legítimo).
- **H4 (adaptabilidad):** agregar un ataque nuevo al KB mejora la detección de su familia **sin reentrenar** — lo que un clasificador fine-tuneado no puede.
- **H5 (robustez temporal):** el **AUC de degradación** de RAGE se mantiene bajo a lo largo de los turnos, mientras el de un guardrail tradicional crece.

9. Costo / beneficio (modelo base = 100%)

Escenario	Costo total con RAGE	Latencia extra
Modelo base caro + RAGE eficiente	~100.5% – 110%	+50–600 ms
Solo embeddings + reglas (sin LLM extra)	~101% – 105%	+50–150 ms
Modelo base barato + clasificador chico	~112% – 130%	+200–500 ms
Peor caso (guard con LLM tan caro como el principal)	~150% – 200%	+0.5–1 s

Además, RAGE **bloquea ataques antes** de llamar al LLM caro → ahorra esas llamadas, compensando parte del overhead. El costo real crítico no es cómputo, sino **latencia y falsos positivos** (el “trilema”).

10. Métricas de evaluación

- **ASR** (Attack Success Rate) global y por categoría OWASP.
- **Score de severidad** 0–100 con bandas (NONE / LOW / ELEVATED / CRITICAL).
- **AUC de degradación** multi-turno (normalizado).
- **Tasa de falsos positivos** sobre tráfico legítimo (utilidad).
- **Latencia y sobrecosto** (% sobre el costo del LLM base).

11. Plan de trabajo (hackathon)

Día	Foco	Entregables
Sábado	Construir el núcleo (tracks en paralelo)	Track A: motor de decisión + gateway · Track B: RAG de amenazas (KB) · Track C: filtro de intención (dinámico). Cierre: integrar tras un LLM real.
Domingo	Demo + evidencia + pulido	Escenario grave (agente + DB falsa, DROP TABLE) · curvas de AUC antes/después · dashboard del ASR cayendo · ensayo del pitch.

Cada track es una rama y un agente de Cursor en paralelo; el **harness** existente es el test de regresión (mide ASR + falsos positivos en cada cambio).

12. Limitaciones y riesgos honestos

- **Ataques adaptativos:** ninguna defensa (RAGE incluido) resiste a un atacante que optimiza contra ella. RAGE reduce el riesgo, no lo elimina.
- **Gap semántico:** el RAG puede fallar ante ataques 0-day que no se parecen a nada del KB.
- **LLM08:** RAGE hereda el riesgo de su propio vector store (envenenamiento del KB).
- **Trilema:** subir la seguridad puede subir los falsos positivos — hay que medirlo.

13. Referencias

- OWASP Top 10 for LLM Applications 2025 — genai.owasp.org/llm-top-10/
- RAD: Retrieval-Augmented Defense — arXiv 2508.16406
- Securing AI Agents Against Prompt Injection (framework multicapa RAG) — arXiv 2511.15759
- StruQ (USENIX Security 25) y SecAlign — arXiv 2410.05451
- The Attacker Moves Second (Nasr et al., 2025).
- Documentos del repo: REPORT.md, docs/owasp-y-estado-del-arte.md, docs/estado-del-arte-deep-research.md